

Event Driven Architecture and Kafka Explained: Pros and Cons

30.6.2023

Event-driven architecture is increasingly preferred over traditional request-response architecture in web-scale application design due to its benefits in complex, large-scale applications. Unlike synchronous communication in request-response architecture, the event-driven architecture allows for asynchronous communication, enabling concurrent and independent operation of components. It supports real-time data processing and facilitates the integration of various systems and services, making them well-suited for complex and dynamic environments.

What is an event?

An event is an action that prompts a notification or changes the application's state. For instance, an event can occur when you post something on social media, place an order online, complete a financial transaction, or register for an account. These events contain relevant contextual information that triggers corresponding actions in other components, such as notifying followers, managing inventory, processing payments, and fulfilling orders.

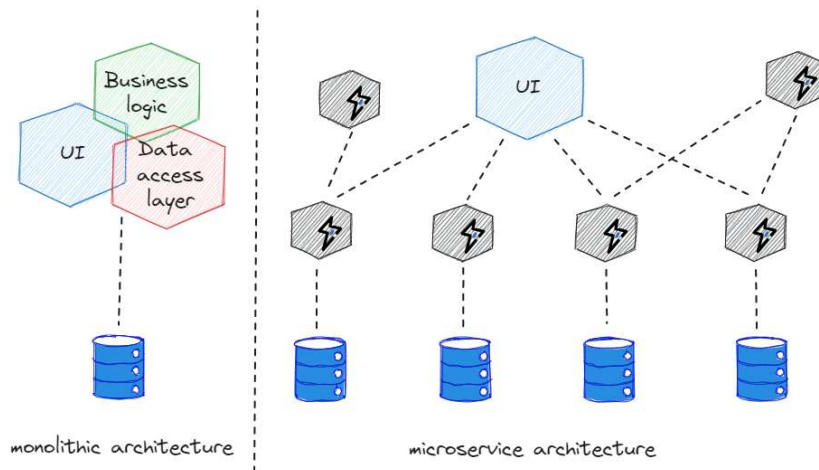
Let's use an example of social media and see the differences between event-driven architecture and request-response architecture.

Event driven architecture Vs. Request-response architecture

When you engage in social media activities in an event-driven architecture, actions such as posting a new message or liking a post generate events that trigger various actions like updating timelines, notifying followers, and storing the content. This real-time event processing enables *immediate updates and interactions across the platform, ensuring a dynamic and engaging user experience.*

On the other hand, in a traditional request-response architecture, interacting with social media involves sending requests to the server for actions like posting or liking. The server processes these requests, updates the necessary data, and responds back to you, indicating the success or failure of the operation. Your browser then waits for the response before updating the interface or displaying relevant notifications. In this case, you might experience *a slight delay as you wait for responses from the server, potentially affecting your perception of speed and interactivity.*

Event driven development: Microservice architecture



In social media, a monolithic architecture would mean that all the functionalities, such as user profiles, news feed, notifications, and messaging, are tightly coupled within a single application. Managing and updating these interconnected functionalities becomes complex and challenging as the social media platform grows.

However, in a microservice architecture with event-driven communication, the functionalities are divided into separate microservices. For example, individual microservices would be for user profiles, news feeds, notifications, and messaging. When you post something, an event is triggered and propagated to the relevant microservices. The microservice responsible for the news feed updates the feed, the notification microservice sends notifications to your followers, and the messaging microservice stores and processes the message. This modular approach allows each microservice to independently handle its specific tasks, enabling easier development, scalability, and maintenance of the social media platform as a whole.

Kafka and Microservices

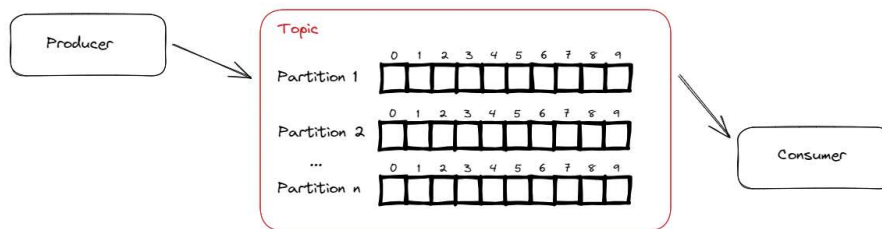
Apache Kafka is like a superhero for microservices, tackling all those pesky issues of orchestration while delivering scalability, efficiency, and lightning-fast speed. It's the go-to tool for inter-service communication, keeping latency super

low and handling failures like a champ. It acts as a central messaging system, enabling seamless data exchange and coordination between different services. Thanks to its horizontal scaling capabilities, you can add more service instances as your workload grows. Plus, its distributed nature ensures high throughput and keeps up with demanding workloads without breaking a sweat.

Efficiency? With its publish-subscribe model, services can publish messages to topics, and other services can subscribe to those topics to get the notifications they need. This decoupled setup lets services evolve independently, keeping the system running smoothly and efficiently.

And let's remember ultra-low latency and fault tolerance. Kafka stores messages in a distributed and replicated manner, guaranteeing data durability even in the face of failures. So if a service or a node goes kaput, the show goes on without losing data or messing up the workflow.

How does Event Driven Architecture work with Kafka?



Producer and Consumer

In an event-driven architecture, events follow a unidirectional flow, moving from a producer to a consumer. *A producer is an entity that generates and publishes events, while a consumer is an entity that subscribes to and processes those events.*

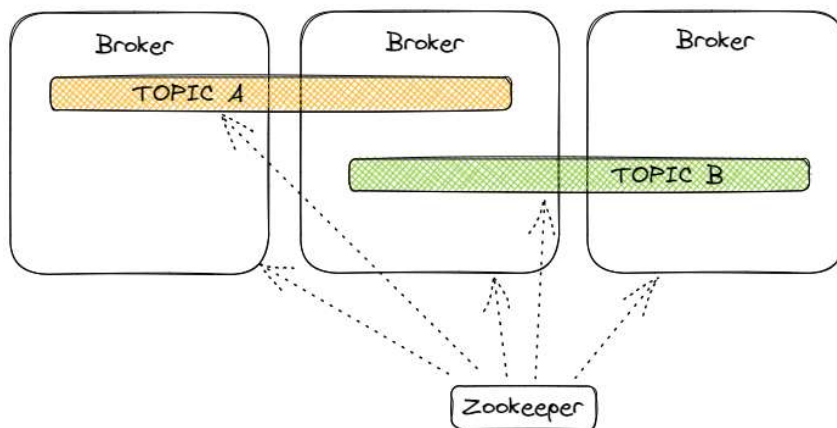
Think of a social media platform where you can post your updates. When you create a new post, you act as the *event producer*. Unlike a phone call, where you expect a direct

response, the event producer, in this case, doesn't anticipate a response from *the consumer*. This asynchronous nature of events eliminates the need to block code execution, avoiding timeouts and ensuring smoother processing.

Events occur due to specific actions, and no predefined target system exists. Instead, services express their interest in the events generated by other services. In our social media example, service B, representing the newsfeed service, would express its interest in the events produced by service A, representing the user posting service. *However, there might be other services, such as service C or D, that are also interested in these events.* This allows for a decoupled architecture where services can react to events independently, enhancing flexibility and promoting scalability.

But how can we ensure that an event triggered by one system reaches all the interested services?

Broker & Zookeeper examples in Kafka: Event driven development



Broker and Zookeeper

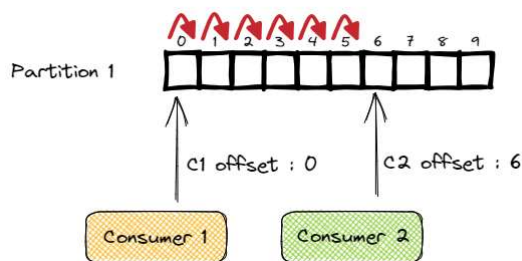
In event-driven development, *message brokers* are essential for decoupling applications and ensuring availability. They serve as *intermediaries between event producers and consumers, allowing scalability by adding more nodes as*

needed. These brokers work together as a distributed system, replicating and delivering messages for consumption.

Zookeeper manages various coordination tasks. It maintains configuration information for the Kafka cluster, including broker locations and topics. It also handles leader elections for partitions, ensuring message replication and availability even during failures. Zookeeper manages consumer offsets, allowing them to resume reading from where they left off after a restart or failure.

In summary, combining brokers and Zookeeper in Kafka enables the social media platform to handle storing, distributing, and coordinating events generated by user activities. This setup provides a reliable and scalable messaging system for the platform's event-driven architecture.

Topic, partition, offset in Kafka



Topic, partition, offset

Topic

In Kafka, a topic is a stream of messages that can be divided and spread across multiple brokers in the cluster. You can have many producers and consumers for a topic, and creating and managing them as needed is easy.

Partition

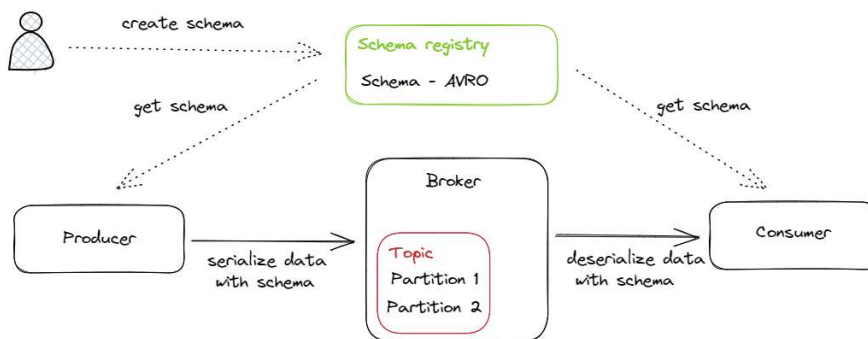
A partition is like a chunk of messages within a topic, which lives on a single broker. Partitions help Kafka handle heavy workloads by splitting the storage and serving of messages

across different brokers. Each partition has a unique ID called a partition key, which helps determine which broker handles the messages in that partition.

Offset

An offset is a special ID assigned to each message in a partition. It shows the exact position of the message within the partition. Consumers can use offsets to track which messages they have already read from a topic, making it easy to pick up where they left off.

Schema registry



Schema registry and Avro

Schema Registry is a centralized service that stores and manages schemas for data serialization in a distributed system. It provides a repository for registering and retrieving schemas, enabling data producers and consumers to ensure compatibility and consistency in their data structure. By managing schemas, the Schema Registry simplifies working with different data formats and versions, facilitating seamless integration and interoperability in data-driven applications.

The schema registry in Kafka is like a handy service that keeps track of Avro schemas. It stores and manages schemas so that producers can register new ones, and consumers can easily access the latest versions when reading messages from a topic. Speaking of **Avro**, it's a nifty format used in **Kafka** to make messages nice and compact. With Avro

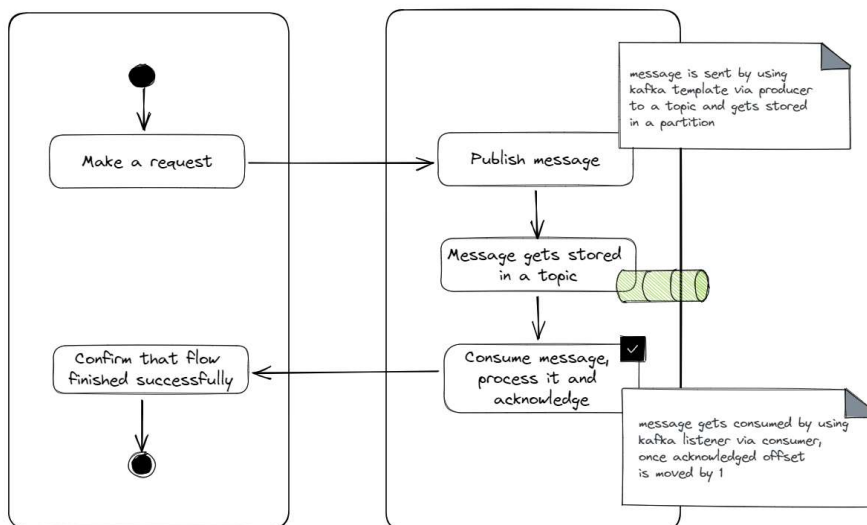
schemas, you can define how your data should look, specifying the types of each field and any nested structures.

When you post a new message or like a post on social media, the data representing that action needs to be encoded and transmitted; this is where Avro's schemas come into play. The Avro schema defines the data structure, specifying the types of each field and any nested structures. For example, the schema may define fields like "user_id," "post_id," "timestamp," and "action_type." The schema registry stores and manages these Avro schemas for the social media platform.

Additionally, the schema registry provides compatibility checks. It ensures that producers and consumers are using compatible versions of the schema. This is important because the schema may change as the social media platform evolves to accommodate new features or data fields. The compatibility checks help prevent issues where a producer sends data that consumers cannot interpret or process correctly due to schema mismatch.

Here is a demo app that can help you better understand the concepts mentioned earlier:

<https://github.com/PRODYNA/insights-kafka-demo>



Pros and Cons of Kafka event driven development

Pros:

Fast and furious: Kafka is a data processing beast, devouring large volumes of data with lightning speed, perfect for real-time data streaming.

Scale it up: Need more power? Its architecture lets you beef up your data handling by adding more brokers and partitions.

Data's best friend: It keeps your data safe and sound, storing it for the long haul. So, go ahead and analyze that treasure trove of information.

Real-time data processing: With Kafka, you can process and analyze data in real-time, giving businesses the edge to make quick, data-driven decisions.

Jack of all trades: Kafka is versatile, fitting in various scenarios like messaging, stream processing, and data integration. It's the Swiss Army knife of data handling.

Cons:

Complexity: Setting up and managing a Kafka cluster can be a brain teaser, demanding profound know-how and constant monitoring.

Hunger Games: It can be a resource hog, especially when dealing with huge data volumes. We optimize resource allocation and use nifty compression techniques to keep things in check.

Data loss scares: While it is built to be tough, there's still a slim chance of data loss if a broker goes haywire. At PRODYNA, we configure replication factors and build fault tolerance to minimize any potential risks.

Integration jigsaw: Integrating it with your existing workflows and tech stack might require tweaking and compatible connectors. Tap into the rich Kafka ecosystem, rely on the supportive community, and harness middleware magic to smooth things out.

Cost: Setting up and managing a Kafka cluster can burn a hole in your pocket. We explore cloud-based Kafka services and intelligent resource allocation to keep those expenses in check. It's all about being savvy with the cash.

With messaging solutions like Kafka, event-driven architecture brings many benefits for building distributed, microservices-based applications. It offers scalability, responsiveness, and resilience, making it an attractive business choice.

As a software architect, I love Kafka and event-driven architecture for several reasons. Firstly, the Kafka community is excellent, providing valuable support and resources. The comprehensive documentation makes it easy to understand and implement Kafka in various projects. Secondly, Kafka offers extensive features and flexibility, allowing me to build scalable and resilient systems tailored to specific requirements. Lastly, the seamless integration and testing capabilities, especially with Java and Spring Boot, make it a pleasure to work with Kafka and ensure the reliability of my applications. The availability of test Kafka containers further simplifies the testing process, making it even more convenient.



**Bojana
Belojevic**

Useful Links:

<https://kafka.apache.org/documentation/>

Software
Architect

<https://docs.spring.io/spring-kafka/reference/html/>

<https://avro.apache.org/docs/1.11.1/>

<https://github.com/confluentinc/examples>

<https://developer.confluent.io/get-started/spring-boot/>

Bojana Belojevic is an experienced software architect with 9 years of expertise. Her focus lies in applying a holistic approach to develop robust software solutions.



More related topics



Software Product Roadmap

Minimum Viable Product OpenAI

Why a Strong Product Roadmap is Key to Successful Software Development

What makes a product roadmap good and valuable? How can you create and maintain it effectively? We explore why it is important and what benefits it drives. Read and discover the best practices for creating and keeping a roadmap vivid and alive.

[Read More](#)

**Scroll to the bottom to return
to the Overview**

